

KOCAELİ ÜNİVERSİTESİ MÜHENDİSLİK FAKÜLTESİ

**Gömülü Sistem Üzerinde Görüntü İşleme Tabanlı Yapay Zeka
Destekli Dost-Düşman Hedef Tespit ve Takip Sistemi**

SUDE ÇAKIR

Bölümü: Elektronik ve Haberleşme Mühendisliği

KOCAELİ, 2025

1. İÇİNDEKİLER

İÇİNDEKİLER	ii
1. GİRİŞ	1
2. PROJEDE YAPILAN ÇALIŞMALAR	4
2.1 Kavramsal Tasarım	4
2.2 İş-Zaman Çizelgesi	5
2.3 Projede Gerçekleşen İş Paketleri ve Ara Çıktıları	5
3. KAYNAKLAR	12

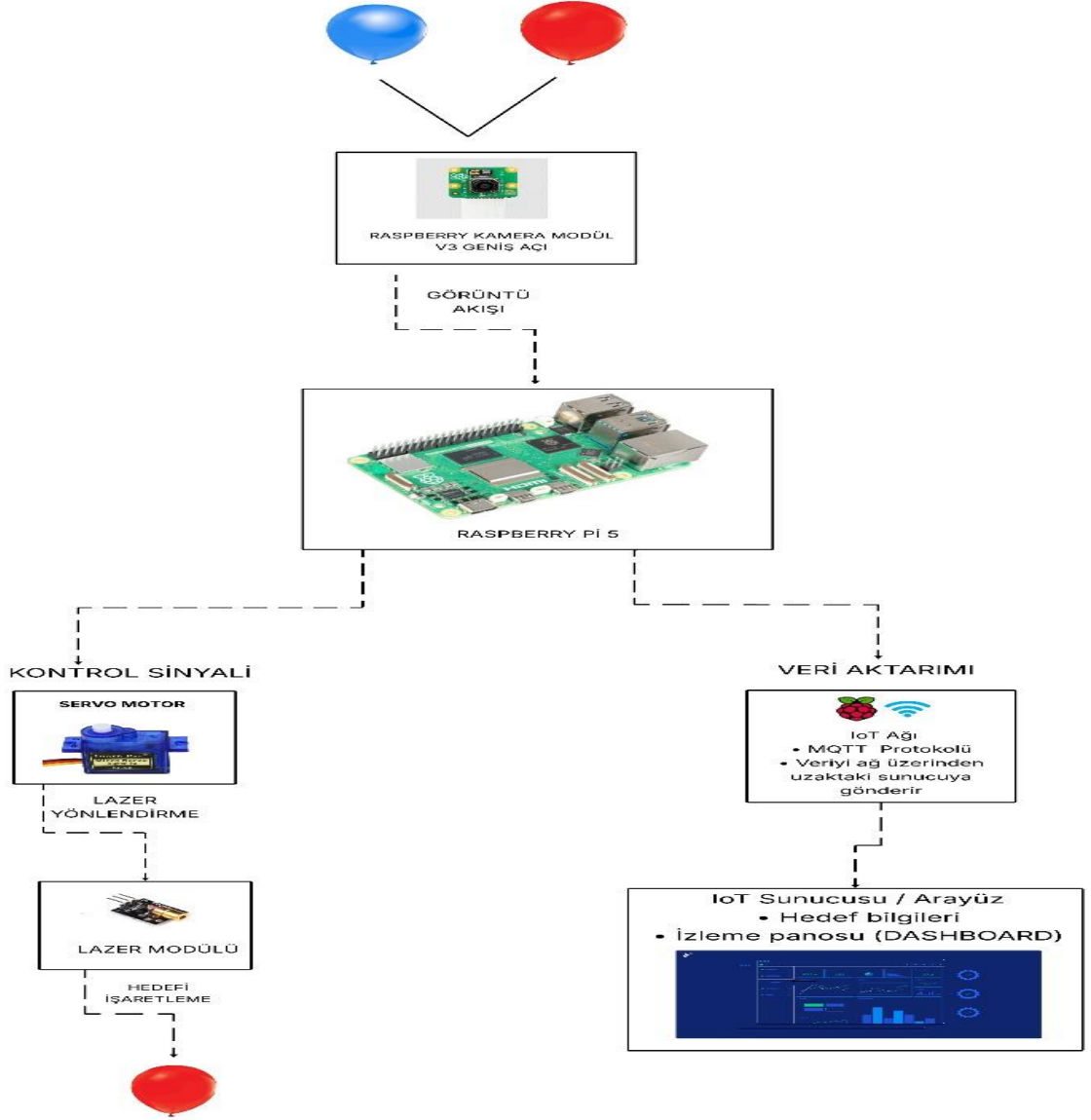
2. GİRİŞ

Günümüzde savunma sistemlerinde hedef tespiti, sınıflandırma ve takip süreçlerinin hızlı, güvenilir ve uzaktan izlenebilir olması büyük önem taşımaktadır. Özellikle otonom gözetleme ve insansız sistemlerde, gerçek zamanlı karar verebilen gömülü yapay zeka çözümlerine olan ihtiyaç hızla artmaktadır. Bu doğrultuda geliştirilen projede, görüntü işleme teknikleri kullanılarak dost–düşman hedef ayrımı yapabilen, tespit edilen düşman hedefi servo kontrollü bir mekanizma ile takip edip merkezleyebilen ve doğrulama sonrası lazer işaretleme simülasyonu gerçekleştirebilen bir gömülü sistem tasarlanması amaçlanmaktadır.

Sistem, kamera modülü üzerinden alınan görüntüyü Raspberry Pi üzerinde çalışan yapay zeka modeliyle analiz etmekte ve tespit edilen nesneleri konum, renk ve sınıf bilgilerine göre değerlendirmektedir. Düşman hedef algılandığında pan–tilt servo mekanizması hedefi görüş hattının merkezine getirmekte ve lazer modülü ile işaretleme yapılmaktadır.

Bu projede geliştirilen sistem yalnızca yerel çalışmayıp, aynı zamanda IoT altyapısına entegre edilerek uzak erişim ve izleme yeteneğine de sahip olacaktır. Tespit edilen hedef türü, konumu, zaman damgası ve sistem durumu gibi bilgiler MQTT tabanlı bir yapıyla buluta aktarılacak; kullanıcılar web veya mobil uyumlu özel olarak tasarlanmış bir arayüz üzerinden sistemi gerçek zamanlı olarak izleyebilecektir. Bu arayüz; hedef tespit geçmişi, anlık görüntü akışı, sistem durumu, servo açılı bilgileri ve lazer aktivasyon kayıtlarını kullanıcıya görsel ve anlaşılır bir formatta sunacaktır.

Bu çalışma; görüntü işleme, gömülü yapay zeka, servo kontrol, IoT mimarisi ve kullanıcı arayüz tasarımı gibi birçok alanı bir araya getirerek kapsamlı bir mühendislik uygulaması ortaya koymaktadır. Projenin uygulama alanları arasında otonom gözetleme sistemleri, güvenlik robotları, insansız araçlar, laboratuvar prototipleri ve savunma teknolojilerinde kullanılan hedef tespit–takip platformları bulunmaktadır.



Şekil 1: Kırmızı-Mavi balon hedef tespit blok gösterimi

Sistemin genel işleyişini göstermekte olup sahnedeki hedeflerin renk temelli yapay zekâ analizi ile ayırt edilmesini temel almaktadır. Diyagramda yer alan mavi ve kırmızı balonlar, proje kapsamında kullanılan dost ve düşman hedef modellerini temsil etmektedir. Mavi balon dost hedef olarak kabul edilirken, kırmızı balon düşman hedef olarak değerlendirilmekte ve sistemin tepkisi yalnızca düşman hedefe karşı tetiklenmektedir.

Sahnedeki balonlar Raspberry Pi Kamera Modülü V3 Geniş Açı tarafından görüntülenmekte ve elde edilen görüntü akışı Raspberry Pi 5'e iletilmektedir. Raspberry Pi üzerinde çalışan yapay zekâ modeli, balonları renk, şekil ve konum bilgileri üzerinden analiz ederek hedef tespiti gerçekleştirmektedir. Bu aşamada sistem, balonun rengine göre dost–düşman sınıflandırmasını gerçek zamanlı olarak yapmaktadır.

Düşman hedef (kırmızı balon) tespit edildiğinde Raspberry Pi, servo motorlara kontrol sinyali göndererek lazer modülünün yönünü hedefe göre ayarlamaktadır. Servo motor, lazeri düşman hedefin merkezine getirecek şekilde konumlandırır ve lazer modülü hedefi işaretler. Böylece sistem, otonom bir hedef tespit–takip ve lazerle işaretleme mekanizması oluşturur.

Ayrıca sistem, yalnızca fiziksel işaretleme yapmayıp tespit edilen dost veya düşman hedef bilgilerini, konum verilerini ve zaman damgalarını IoT altyapısı üzerinden uzak bir sunucuya iletmektedir. Bu amaçla MQTT protokolü kullanılmakta ve veriler kablosuz ağ üzerinden IoT sunucusuna aktarılmaktadır. Kullanıcılar, özel olarak tasarlanmış bir izleme arayüzü (dashboard) üzerinden hedef tespit geçmişini, anlık sistem durumunu, tespit edilen balonun türünü (kırmızı/mavi), konum bilgisini ve sistem aktivitelerini gerçek zamanlı olarak takip edebilmektedir.

Bu yapı sayesinde sistem hem laboratuvar ortamında dost–düşman ayrımı yapabilen otonom bir fiziksel prototip olarak çalışmakta hem de IoT entegrasyonu sayesinde uzaktan izlenebilir bir güvenlik platformu haline gelmektedir.

3. PROJEDE YAPILAN ÇALIŞMALAR

3.1 Kavramsal Tasarım

Projenin ilk aşamasında, dost ve düşman hedeflerinin tanınması için kullanılacak veri tabanının oluşturulmasına başlanmıştır. Bu kapsamda kırmızı ve mavi balonlara ait çeşitli açılardan, farklı ışık koşullarında ve farklı arka planlarda çekilmiş görüntüler toplanmış; bu görüntüler Roboflow platformuna yüklenerek etiketleme ve ön işleme süreçleri uygulanmıştır. Veri kümesinin çeşitliliğinin artırılması ve model eğitime uygun hâle getirilmesi için augmentasyon ve optimizasyon çalışmaları devam etmektedir. Hazırlanan veri kümesi ilerleyen aşamalarda YOLOv8 modelinin eğitiminde kullanılmaktadır.

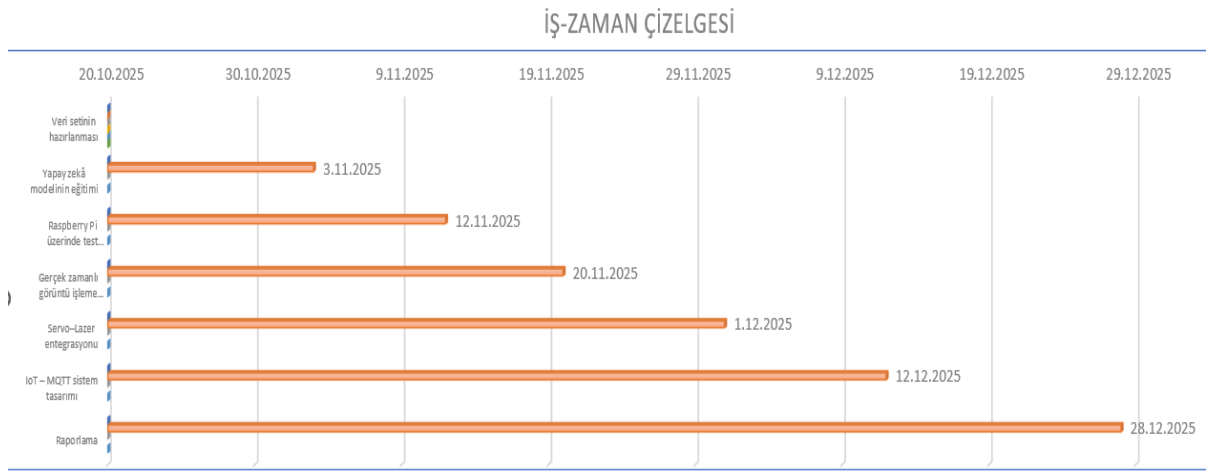
Bu veri kümesi ile YOLOv8 modeli üzerinde birden fazla eğitim denemesi yapılmış ve ilk test sonuçları incelenmiştir. Farklı epoch, batch size ve veri artırma yöntemleri denenerek model performansının artırılması hedeflenmiştir. Eğitim süreci hâlâ geliştirme aşamasındadır ve performans analizleri doğrultusunda veri kümesinin yeniden düzenlenmesi veya ek görüntülerle desteklenmesi planlanmaktadır. Bu sayede kırmızı ve mavi balonların doğru sınıflandırılmasını sağlayacak daha kararlı bir yapay zekâ modeli oluşturulacaktır.

Elde edilen modeller ilerleyen aşamada Raspberry Pi 5 üzerinde gerçek zamanlı olarak test edilmiştir. Raspberry Pi kamera modülü ile alınan görüntülerin canlı olarak modele aktarılması, modelin sahnedeki balonları tespit edip sınıflandırması ve sistemin kare bazlı performansının ölçülmesi bu aşamada gerçekleştirilmiştir. Gerçek zamanlı testlerde servo motorların hedefe yönelmesi ve lazer modülünün düşman balonu işaretlemesi gibi fiziksel kontrol süreçleri de değerlendirilecektir. Bu aşama hâlen hazırlık sürecinde olup testler raporun ilerleyen dönemlerinde gerçekleştirilecektir.

Son aşamada sistemden elde edilen hedef bilgileri IoT altyapısına aktarılacak ve MQTT protokolü ile uzak bir sunucuya gönderilecektir. Bu kapsamda geliştirilecek dashboard arayüzünde dost ve düşman hedeflerin gerçek zamanlı gösterimi, tespit

geçmişi ve sistem durumu kullanıcıya sunulacaktır. IoT entegrasyonu şu anda planlama aşamasında olup gerekli yazılım ve sunucu düzenlemeleri tamamlandıktan sonra hayata geçirilecektir. Projenin genel akışı kapsamında veri seti oluşturma, model eğitimi, gerçek zamanlı testler ve IoT aktarımı aşamaları adım adım ilerletilmekte olup dönem sonunda tamamlanması hedeflenmektedir.

3.2 İş-Zaman Çizelgesi

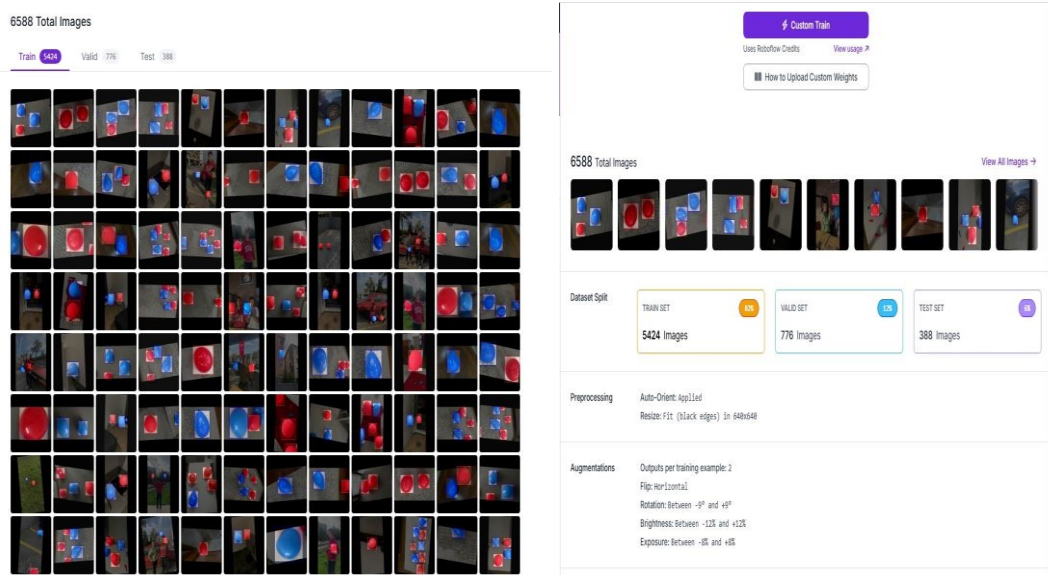


Şekil 2: İş-Zaman Çizelgesi Gantt Şeması

3.3 Projede Gerçekleşen İş Paketleri ve Ara Çıktıları

Projenin ara rapor dönemine kadar olan süreçte çalışma planına uygun şekilde ilerlenmiş ve proje takviminde belirtilen iş paketlerinin büyük bölümü başarıyla tamamlanmıştır. İlk aşamada proje için gerekli özgün veri seti oluşturulmuş; farklı açılardan, farklı ışık koşullarında ve çeşitli ortamlarda çekilmiş kırmızı ve mavi balon görüntüleri toplanarak Roboflow platformunda etiketlenmiş ve artırılmıştır. Elde edilen veri kümesi YOLOv8 yapay zekâ modelinin eğitiminde kullanılmış, model birden fazla kez eğitilerek farklı epoch ve parametre kombinasyonları denenmiş ve performansı iyileştirilmiştir. Eğitim aşamasından sonra model Raspberry Pi 5 üzerinde test edilmeye başlanmış, gerçek zamanlı görüntü işleme denemeleri yapılmış ve kamera aracılığıyla alınan karelerde balon tespiti ve sınıflandırma doğrulaması gerçekleştirilmiştir. Raspberry Pi'nin donanım kurulumu, kütüphane yüklemeleri ve test ortamının hazırlanması başarıyla tamamlanmıştır. Bu

aşamada servo ve lazer modülü entegrasyonu için altyapı çalışmaları yapılmış olup mekanik yönlendirme testlerinin hazırlıkları devam etmektedir. Genel olarak projenin mevcut durumu iş-zaman çizelgesi ile uyumludur; veri seti oluşturma, model eğitimi ve gerçek zamanlı analiz gibi kritik iş paketleri planlanan süreler içinde tamamlanmış, sonraki aşamalar için gerekli zemin oluşturulmuştur. Bu süreçte elde edilen ara çıktılar, projeye özel veri seti, eğitilmiş YOLOv8 modeli, ilk gerçek zamanlı tespit denemeleri ve Raspberry Pi test ortamının oluşturulması şeklindedir. İlerleyen dönemlerde servo-lazer kontrolünün tamamlanması, IoT-MQTT altyapısının kurulması ve dashboard arayüzünün geliştirilmesi planlanmaktadır.



Şekil 3: Veri Setinden Örnek Görseller

Proje kapsamında oluşturulan kırmızı ve mavi balon veri setine ait örnek görüntüler. Veri seti farklı ışık koşullarında, farklı açılarda ve farklı ortam çeşitliliği sağlanarak oluşturulmuştur.

```
!pip install roboflow

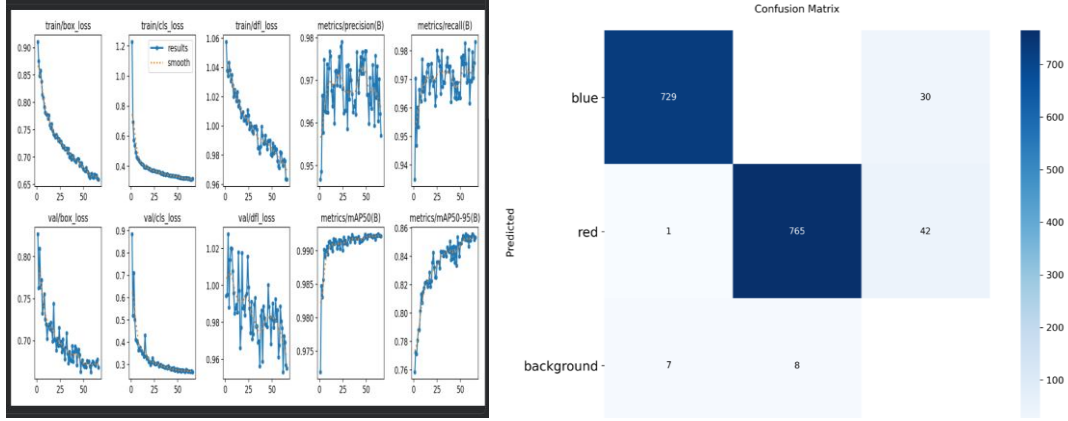
from roboflow import RoboFlow
rf = RoboFlow(api_key="wGV0mHPPr1CABg2IzqW0")
project = rf.workspace("sude-1n3gj").project("red-blue-balloon-detection-yq5xy")
version = project.version(1)
dataset = version.download("yolov8")

Requirement already satisfied: roboflow in /usr/local/lib/python3.12/dist-packages (1.2.11)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from roboflow) (2025.10.5)
Requirement already satisfied: idna==3.7 in /usr/local/lib/python3.12/dist-packages (from roboflow) (3.7)
Requirement already satisfied: cyciler in /usr/local/lib/python3.12/dist-packages (from roboflow) (0.12.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from roboflow) (1.4.9)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from roboflow) (3.10.0)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from roboflow) (2.0.2)
Requirement already satisfied: opencv-python-headless==4.10.0.84 in /usr/local/lib/python3.12/dist-packages (from roboflow) (4.10.0.84)
Requirement already satisfied: Pillow>=7.1.2 in /usr/local/lib/python3.12/dist-packages (from roboflow) (11.3.0)
Requirement already satisfied: pi-heif<2 in /usr/local/lib/python3.12/dist-packages (from roboflow) (1.1.1)
Requirement already satisfied: pillow-avif-plugin<2 in /usr/local/lib/python3.12/dist-packages (from roboflow) (1.5.2)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.12/dist-packages (from roboflow) (2.9.0.post0)
Requirement already satisfied: python-dotenv in /usr/local/lib/python3.12/dist-packages (from roboflow) (1.1.1)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from roboflow) (2.32.4)
Requirement already satisfied: six in /usr/local/lib/python3.12/dist-packages (from roboflow) (1.17.0)
Requirement already satisfied: urllib3>=1.26.6 in /usr/local/lib/python3.12/dist-packages (from roboflow) (2.5.0)
Requirement already satisfied: tqdm>=4.41.0 in /usr/local/lib/python3.12/dist-packages (from roboflow) (4.67.1)
Requirement already satisfied: PyYAML>=5.3.1 in /usr/local/lib/python3.12/dist-packages (from roboflow) (6.0.3)
Requirement already satisfied: requests-toolbelt in /usr/local/lib/python3.12/dist-packages (from roboflow) (1.0.0)
Requirement already satisfied: filetype in /usr/local/lib/python3.12/dist-packages (from roboflow) (1.2.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->roboflow) (1.3.3)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->roboflow) (4.60.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->roboflow) (25.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->roboflow) (3.2.5)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->roboflow) (3.4.4)
loading RoboFlow workspace...
loading RoboFlow project...
Downloading Dataset Version Zip in "Red-Blue-Balloon-Detection"-1 to yolov8:: 100%|██████████| 158501/158501 [00:01<00:00, 92422.80it/s]

Extracting Dataset Version Zip to "Red-Blue-Balloon-Detection"-1 in yolov8:: 100%|██████████| 9796/9796 [00:01<00:00, 9384.14it/s]
Creating new Ultralytics Settings v0.0.6 file ✓
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
```

Şekil 4: Roboflow API ile Veri Setinin Programatik Olarak İşlenmesi

Veri seti Roboflow üzerinde etiketlendikten sonra Python tabanlı API entegrasyonu kullanılarak YOLOv8 formatında sisteme indirilmiştir. Bu işlemde Roboflow'un sağladığı API anahtarı ile proje workspace'i tanımlanmış, veri setinin ilgili sürümü seçilmiş ve model eğitiminde kullanılmak üzere dosyalar otomatik olarak hazırlanmıştır. Notebook ortamında başlatılan süreçte gerekli tüm kütüphanelerin başarıyla yüklendiği ve veri setinin eğitim, doğrulama ve test klasörlerine ayrılmış şekilde indirildiği doğrulanmıştır. Bu adım, yapay zekâ modelinin eğitime başlanabilmesi için gerekli altyapının oluşturulduğunu göstermektedir.



Şekil 5: YOLOv8 eğitim sonuçları grafikleri

Modelin sınıflandırma performansını gösteren confusion matrix incelendiğinde, kırmızı ve mavi balonların yüksek doğrulukla tespit edildiği görülmektedir. Mavi balon sınıfında 729, kırmızı balonda ise 765 doğru sınıflandırma yapılmış olup yanlış pozitif ve yanlış negatif değerlerin oldukça düşük seviyede kalması modelin renk temelli dost-düşman ayrımını güvenilir şekilde gerçekleştirdiğini göstermektedir.

YOLOv8 modelinin eğitim süreci boyunca elde edilen kayıp ve performans grafiklerinde, hem eğitim hem de doğrulama kayıplarının düzenli olarak azaldığı, precision ve recall değerlerinin yüksek seviyelerde seyrettiği ve mAP skorlarının her epoch ile birlikte arttığı görülmektedir. Bu sonuçlar, modelin istikrarlı şekilde öğrenmeye devam ettiğini ve balon tespit performansının başarılı şekilde iyileştiğini göstermektedir.

```

Ultralytics 8.3.221 Python-3.12.12 torch-2.8.0+cu126 CUDA:0 (NVIDIA A100-SXM4-40GB, 40507MiB)
Model summary (fused): 72 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs
val: Fast image access (ping: 0.0±0.0 ms, read: 1145.4±321.3 MB/s, size: 37.9 KB)
val: Scanning /content/Red-Blue-Balloon-Detection-1/valid/labels.cache... 577 images, 0 backgrounds, 0 corrupt: 100% 577/577 1.2Mit/s 0.0s

```

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95
all	577	1510	0.963	0.975	0.992	0.855
blue	489	737	0.969	0.977	0.993	0.854
red	483	773	0.958	0.974	0.991	0.856

```

Speed: 1.3ms preprocess, 1.2ms inference, 0.0ms loss, 0.7ms postprocess per image
Results saved to /content/runs/detect/val

=== VALIDATION SONUÇLARI ===
Precision: 0.992
Recall: 0.855
mAP@50: 0.992
mAP@50-95: 0.973

```

Şekil 6: YOLOv8 validasyon performans sonuçları

Validasyon sonuçları incelendiğinde, YOLOv8 modelinin kırmızı ve mavi balonları yüksek doğrulukla tespit ettiği görülmektedir. Model genel olarak %0.992 precision,

%0.855 recall, %0.992 mAP@50 ve %0.973 mAP@50–95 değerlerine ulaşmıştır. Sınıflar bazında da box, precision ve recall metriklerinin tutarlı olduğu, her iki sınıf için de mAP değerlerinin yüksek seviyede seyrettiği görülmektedir. Bu sonuçlar, oluşturulan veri setinin model tarafından başarıyla öğrenildiğini ve sistemin gerçek zamanlı testler için yeterli performansa sahip olduğunu göstermektedir.

Model eğitimi başarıyla tamamlandıktan sonra bir sonraki aşama, eğitilen YOLOv8 modelinin hem kayıtlı video üzerinde hem de gerçek zamanlı kamera akışında test edilmesidir. Bu aşamada amaç, modelin gerçek sahne koşullarında nasıl davrandığını, tespit hızını, doğruluk seviyesini ve farklı ışık–açı değişimlerine karşı tepkisini gözlemlemektir. Video testlerinde kırmızı ve mavi balonların doğru şekilde algılanıp algılanmadığı, tespit kutularının konum isabeti, sınıf etiketlerinin doğruluğu ve modelin kare başına işleme süresi değerlendirilir. Video üzerinde yapılan bu ilk testler, modelin Raspberry Pi üzerinde kullanılacak gerçek zamanlı sisteme aktarılmadan önce performansının kontrol edilmesini sağlar. Bu nedenle video tabanlı test aşaması, modelin gerçek zamanlı gömülü sisteme entegrasyonu için kritik bir ara değerlendirme süreci olarak uygulanmaktadır.

Modelin bilgisayar ortamındaki video testlerinden başarılı sonuçlar alınmasının ardından, sistemin gömülü platformda çalışmasını sağlamak amacıyla YOLOv8 modeli Raspberry Pi 5'e aktarılmıştır. Bu aşamada önce Raspberry Pi'nin yazılımsal gereksinimleri (Python, OpenCV, Ultralytics, donanım sürücüler ve kamera modülü yapılandırması) kurulmuş, ardından model ağırlıkları cihaz üzerine yüklenmiştir. Raspberry Pi Kamera Modülü V3 Wide Angle aktif edilerek gerçek zamanlı görüntü akışı alınmış ve eğitilmiş model bu akış üzerinde test edilmiştir. Gerçek zamanlı tespit aşamasında sistemin kare başına işlem süresi, FPS değeri, farklı ışık ve açı durumlarındaki performansı ve balon tespit doğruluğu gözlemlenmiştir. Yapılan ilk testlerde modelin kırmızı ve mavi balonları gömülü sistem üzerinde de başarılı şekilde ayırt ettiği görülmüş, böylece servo–lazer yönlendirme aşamasına geçilebilmesi için gerekli altyapı doğrulanmıştır. Raspberry Pi üzerinde

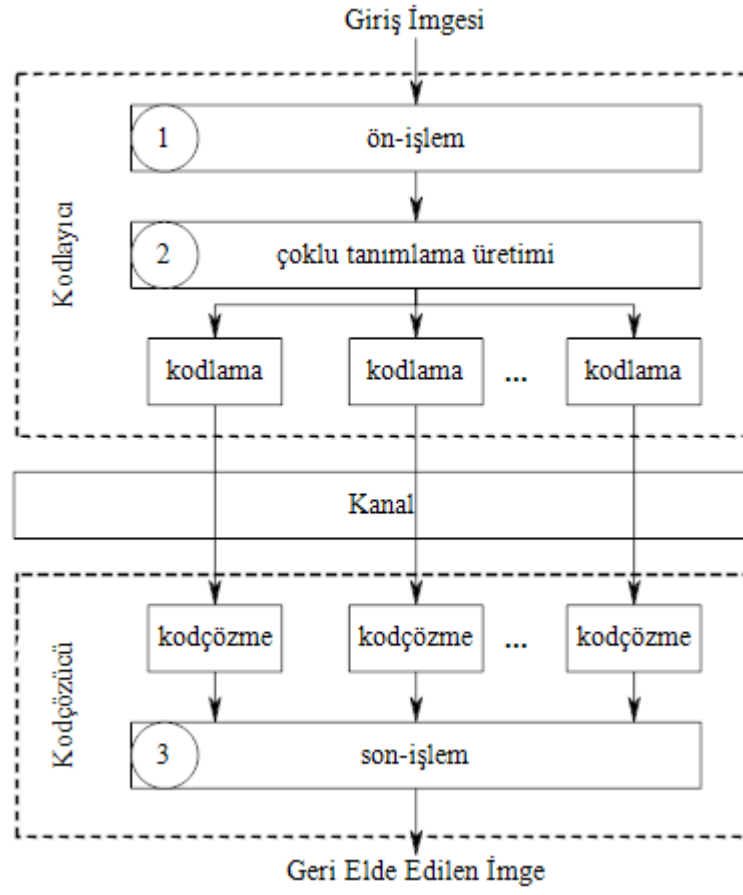
gerçekleştirilen bu gerçek zamanlı tespitler, projenin gömülü sistem kısmının doğru şekilde ilerlediğini ve iş-zaman planıyla uyumlu şekilde sürdüğünü göstermektedir.

[Test1.mp4 - Google Drive](#)

[Test2.mp4 - Google Drive](#)

Not: Bu şablon kullanılırken yeni başlık tanımlamaları, şekil ve tablo yazıları Word'ün Home kısmındaki Heading atamaları kullanılarak oluşturulduğu takdirde, sayfa listesine sağ klik yapılarak otomatik olarak güncelleme sağlanmaktadır. Metin Times New Roman formatında 12 punto ve iki yana yaslı yazılmalı, tablo ve şekil yazıları ise ortalanmış şekilde olmalıdır. Bölüm numaraları yeni sayfada başlamalıdır ve bunun için Word'ün sayfa kesmesi (page break) özelliğinden faydalanılmalıdır.

Kaynaklara atıf yapılırken tek kaynak için [1], ardışık kaynaklar için [1-4] gibi kullanım yapılmalıdır. Aşağıda örnek bir şekil, ilgili şekil yazısı ve şekil yazısına yapılmış olan bir atıf görülmektedir.



Şekil 3.1: ÇTK tabanlı aşırı örneklemenin blok gösterimi [2]

4. KAYNAKLAR

- [1] Goyal, V.K., “Multiple Description Coding: Compression Meets the Network”, *IEEE Signal Processing Magazine*, 18(5), 74-93, (2001).
- [2] Goyal, V.K. “Single and Multiple Description Transform Coding with Bases and Frames”, *In Proc. of SIAM*, 2001.